

Vietnam National University, Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering



DISCRETE STRUCTURE (CO1007)

Assignment Report

A* ALGORITHMS

Student ID: 2510117

Full Name: Lam Duc Anh Khoa

Class: MT25KHM2

Instructor(s): Nguyen An Khuong *CSE-HCMUT*
Le Hong Trang, *CSE-HCMUT*
Tran Tuan Anh, *CSE-HCMUT*
Pham An Vinh, *CSE-HCMUT*
Mai Xuan Toan, *CSE-HCMUT*
Tran Hong Tai, *CSE-HCMUT*

Ho Chi Minh City, May 22, 2026

Contents

1	Introduction	3
1.1	What is A* algorithm?	3
1.2	Fundamental Analysis	3
1.2.1	The Intelligence of A*	3
1.2.2	Pseudocode for A* Tree Search	4
1.2.3	Core Principles of A*	4
1.2.4	Heuristic Function	4
1.3	Implement Initial Components	5
2	Algorithm Analysis and Design	6
2.1	Task 1 — Degrees of Separation in a Social Network	6
2.1.1	How $h(n)$ is Calculated	6
2.1.2	Step-by-Step BFS Propagation	7
2.1.3	Implementation	8
2.2	Task 2 — Drone Delivery in 2D Space with Three Heuristic	13
2.2.1	Problem Overview	13
2.2.2	Source Code	14
2.2.3	Step-by-Step Manual Trace	15
2.2.4	Final Reconstructed Output Path	16
2.2.5	Analytical Comparison: Structural Advantages of Each Heuristic Mode	17
2.3	Task 3 — Warehouse Robot Navigation with Obstacles	18
2.3.1	Problem Overview	18
2.3.2	Heuristic Modes	18
2.3.3	Manual Example Run	18
2.3.4	Source Code	20
2.4	Task 4 — Evacuation Route Planning	23
2.4.1	Problem Overview	23
2.4.2	Grid to Weight Matrix Conversion	23
2.4.3	Manual Example Run	23
2.4.4	Implementation	26
2.4.5	The advantages of implementation in Task 4	29
	References	30

ABSTRACT

This report presents the design, implementation, and analysis of the **A*** (**A-star**) **search algorithm** applied to four practical pathfinding problems as part of the CO1007 Discrete Structure course assignment.

The A* algorithm combines the actual cost $g(n)$ from the start node with a heuristic estimate $h(n)$ to the goal, forming the evaluation function $f(n) = g(n) + h(n)$. When the heuristic is *admissible* (i.e., never overestimates the true cost), A* is guaranteed to find an optimal solution.

The four tasks covered in this report are:

1. **Degrees of Separation in a Social Network** — Finding the shortest connection path between two people in a directed social graph, using a reverse-BFS heuristic.
2. **Drone Delivery in 2D Space** — Finding the shortest delivery route using three distance heuristics: Manhattan, Euclidean, and Chebyshev.
3. **Warehouse Robot Navigation with Obstacles** — Navigating a robot through a 2D grid with walls, supporting 8-directional movement and two heuristic modes.
4. **Evacuation Route Planning** — Converting a floor plan grid into a weighted adjacency matrix and applying A* to find the shortest evacuation path.

For each task, this report explains the cost function design, heuristic selection, and provides a manual step-by-step example run demonstrating the algorithm's behavior.

All implementations are written in C++ using only the standard libraries permitted by the LMS CodeRunner environment: `<iostream>`, `<fstream>`, `<string>`, `<cmath>`, `<vector>`, and `<algorithm>`.

Keywords: A* algorithm, pathfinding, heuristic search, graph theory, shortest path, admissible heuristic, BFS, grid navigation.

1 Introduction

1.1 What is A* algorithm?

A* is a graph traversal and pathfinding algorithm that is applied in many fields in life because of its completeness and optimal efficiency. Being claimed as an informed search algorithm or a best-first search, it is formulated in terms of weighted graphs: starting from a specific starting node of a graph, it aims to find a path to the given node having smallest cost (least distance travelled, shortest time,...).

A* combines the strengths of Dijkstra's algorithm and Greedy Best-First Search by using a cost function defined as:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the actual cost from the start node to the current node
- $h(n)$ is the heuristic estimate of the cost from the current node to the goal
- $f(n)$ is the total estimated cost of the cheapest solution through node n .

1.2 Fundametal Analysis

1.2.1 The Intelligence of A*

Informally speaking, A* Search algorithms, unlike other traversal techniques, possess what can be described as “brains.” This intelligence separates A* from conventional algorithms by allowing it to make informed decisions rather than exploring paths blindly. The efficiency of this approach is why many modern applications, such as video games and web-based maps, utilize A* to find optimal paths across complex environments.

Informal Explanation:

The efficiency of A* comes from balancing two different strategies:

- $g(n)$ (**The Precise**): Exhaustively searches all paths to guarantee the shortest one. It is accurate but computationally slow.
- $h(n)$ (**The Intuitive**): Heuristically "guesses" the distance to the goal. It is very fast but "blind" to obstacles.

A* is considered “smart” because it uses $h(n)$ to prioritize the direction of the goal while $g(n)$ ensures the path remains optimal.

The A* algorithm is a heuristic-based search strategy that enhances efficiency by estimating the cost to reach the goal. It combines the actual distance from the start with an estimated distance to the destination.

1.2.2 Pseudocode for A* Tree Search

Algorithm 1: Structure of the A* Tree Search Algorithm

Input: A search problem and an empty fringe (priority queue)

Output: A solution path or failure

```

fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe);
while fringe is not EMPTY do
    node ← REMOVE-BEST(fringe); ;    // Extract node with lowest f(n)
    if GOAL-TEST[problem] applied to STATE[node] succeeds then
        | return SOLUTION(node);
    end
    successors ← EXPAND(node, problem);
    fringe ← INSERT-ALL(successors, fringe);
end
return failure;
  
```

1.2.3 Core Principles of A*

To implement the "Best-First" strategy, A* relies on the following key components:

- **Evaluation Function:** Each state n is assigned a score using:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$: The exact cost incurred to reach node n from the starting point.
- $h(n)$: The estimated cost (heuristic) from n to the goal.
- **Priority Queue (Fringe):** The *fringe* is typically implemented as a **Min-Heap**. This allows the algorithm to efficiently sort states and always expand the node with the minimum $f(n)$ value first.

1.2.4 Heuristic Function

A critical property for the optimality of the A* algorithm is the use of an **admissible heuristic**.

- Definition and Formal Constraints

A heuristic function $h(n)$ is considered **admissible** if it never overestimates the actual cost to reach the goal state. This means the heuristic is always *optimistic* about the remaining distance. Formally, $h(n)$ must satisfy:

1. $h(n) \leq h^*(n)$, where $h^*(n)$ represents the true minimum cost from node n to the goal.
 2. $h(n) \geq 0$, and specifically $h(G) = 0$ for any goal state G .
- Common Admissible Heuristics for Grid-based Search

Depending on the allowed movement in the floor plan, different distance metrics are employed. These metrics are typically admissible because they represent the "ideal" path without obstacles.

- **Manhattan Distance**

Used when movement is restricted to four directions (Up, Down, Left, Right).

$$h_{Manhattan}(n) = |x_n - x_{goal}| + |y_n - y_{goal}|$$

- **Euclidean Distance**

The straight-line distance between two points. This is the most "optimistic" heuristic as no path can be shorter than a straight line. Used for arbitrary angle or birds-eye movement.

$$h_{Euclidean}(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$

- **Chebyshev Distance**

Used when 8-directional movement is allowed (including diagonals) and the cost of a diagonal move is the same as a horizontal/vertical move.

$$h_{Chebyshev}(n) = \max(|x_n - x_{goal}|, |y_n - y_{goal}|)$$

Source code:

```

1 double getHeuristic(int mode, int x1, int x2, int y1, int y2){
2     if (mode == 1){return abs(x1 - x2) + abs(y1 - y2);}
3     if (mode == 2){return sqrt((x1 - x2)*(x1 - x2) + (y1 - y2)*(
4         y1 - y2));}
5     if (mode == 3){return max(abs(x1 - x2), abs(y1 - y2));}
6     return 0;
7 }
```

Listing 1: Heuristic Function

1.3 Implement Initial Components

** File PathNode.h PathNode.cpp*

```

1 //File PathNode.cpp is empty!
2 #ifndef PATHNODE_H
3 #define PATHNODE_H
4
5 #include <string>
6 struct PathNode {
7     std::string name;
8     double f, g, h;
9     PathNode* next;
10     PathNode() : name(""), f(0), g(0), h(0), next(nullptr){}
11     PathNode(std::string n, double _f, double _g, double _h)
12         : name(n), f(_f), g(_g), h(_h), next(nullptr){}
13 };
14 #endif
```

Listing 2: Heuristic Function

** File Algo.cpp*

```

1 //File PathNode.cpp is empty!
2 #ifndef ALGO_H
3 #define ALGO_H
4
5 #include "PathNode.h"
6 #include <vector>
7 #include <string>
8 #include <cmath>
9 #include <algorithm>
10 #include <fstream>
11 #include <iostream>
12 using namespace std;
13 // Task 1
14 PathNode* findSocialPath(double adjMatrix[100][100], int
    startPerson, int goalPerson);
15 vector<int> calcHeuristic(double adjMatrix[100][100], int goal,
    int n);
16 PathNode* buildPath(const vector<int> &parent, const vector<
    double> &gCost, const vector<double> &hCost, int start, int
    goal);
17 // Task 2
18 PathNode* findDronePath(double adjMatrix[100][100], int coords
    [100][2], int startPoint, int goalPoint, int mode);
19 double getHeuristic(int mode, int x1, int x2, int y1, int y2);
20 // Task 3
21 PathNode* findWarehousePath(int warehouse[100][100], int m, int
    n, int startX, int startY, int goalX, int goalY, int mode);
22 // Task 4
23 PathNode* findEvacuationPath(int floorPlan[100][100], int m, int
    n, int startX, int startY, int exitX, int exitY, double
    weightMatrix[100][100], int mode);
24 #endif

```

Listing 3: Heuristic Function

2 Algorithm Analysis and Design

2.1 Task 1 — Degrees of Separation in a Social Network

2.1.1 How $h(n)$ is Calculated

In Task 1, the graph models a social network where each person is a vertex and each friendship is a directed edge. Since edge weights are uniform (every connection counts as 1 step), the heuristic $h(n)$ is defined as:

The minimum number of connections still needed to reach the goal person from the current person.

There are so many ways to calculate $h(n)$ with consistent result. In this document, we perform a **reverse BFS** from the goal vertex g along the transpose of the adjacency

matrix. This yields the shortest-hop distance from every node to g in the original directed graph.

$$h(n) = \begin{cases} 0 & \text{if } n = \text{goal} \\ d_{\text{BFS}}(n, \text{goal}) & \text{otherwise} \end{cases}$$

Because BFS on an unweighted graph returns the true shortest path, $h(n)$ **never overestimates** the true cost — it is **admissible**, and A* is therefore guaranteed to find an optimal path.

2.1.2 Step-by-Step BFS Propagation

Consider a social network graph where we want to find the path to **Node 4 (Goal)**. The BFS traverses backward to assign distance levels:

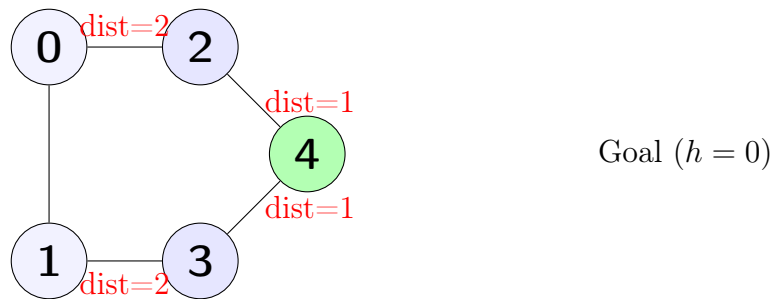


Figure 1: Heuristic values (h) propagation from the Goal node.

Then the distances obtained from BFS are stored in the $h\text{Dist}$ vector used as $h(n)$ values in the A^* evaluation function

Because BFS on an unweighted graph returns the exact shortest path, $h(n)$ is **admissible** (never overestimates), guaranteeing that A* finds the optimal solution.

- Manual Example

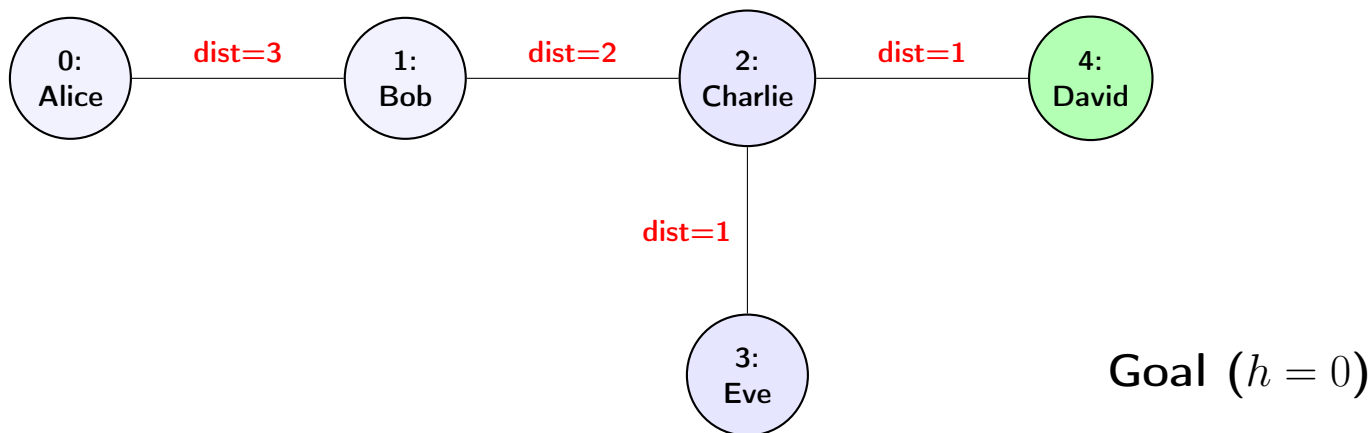


Figure 2: Đồ thị phân bổ tính toán Heuristic (Đích là 4: Eve nm đi Charlie).

Consider a social network with 5 people:

From	To	Edge
0 (Alice)	1 (Bob)	directed
1 (Bob)	2 (Charlie)	directed
2 (Charlie)	3 (David)	directed

Start = 0 (Alice), **Goal** = 3 (David).

Step 1 — Compute $h(n)$ via Reverse BFS from Goal = 3

Node	Person	$h(n)$
0	Alice	3
1	Bob	2
2	Charlie	1
3	David	0

Step 2 — A* Execution

Step	Node opened	$g(n)$	$h(n)$	$f(n)$	openList after
1	0 (Alice)	0	3	3	{1: f=3}
2	1 (Bob)	1	2	3	{2: f=3}
3	2 (Charlie)	2	1	3	{3: f=3}
4	3 (David)	3	0	3	GOAL reached

Result

Solution Path:

Node	f	g	h
0 (Alice)	3	0	3
1 (Bob)	3	1	2
2 (Charlie)	3	2	1
3 (David)	3	3	0

The shortest connection path is: Alice $\rightarrow$ Bob $\rightarrow$ Charlie $\rightarrow$ David (3 degrees of separation).

2.1.3 Implementation

```

1 struct AStarNode1 {
2     int id;
3     double f, g, h;
4     int parent;
5 };

```

Each node stores its graph index, cost values f , g , h , and the index of its parent node for path reconstruction.

- **Heuristic Calculation — Reverse BFS**

Since the social network is a **directed graph**, $h(n)$ cannot be computed by a standard forward BFS from the goal. Instead, we perform a **reverse BFS** by traversing edges backwards:

```
1 if (adjMatrix[next][curr] != 0 && dist[next] == -1) {
2     dist[next] = dist[curr] + 1;
3 }
```

By checking `adjMatrix[next][curr]` instead of `adjMatrix[curr][next]`, we find all nodes that can *reach* `curr` — effectively walking the graph in reverse. This gives the exact shortest-hop distance from every node to the goal, ensuring $h(n)$ is **admissible**.

- **A* Main Loop**

The algorithm maintains an `openList` and a `closed` array. At each iteration, the node with the smallest f is selected:

```
1 for (int i = 1; i < openList.size(); ++i)
2     if (openList[i].f < openList[bestIdx].f)
3         bestIdx = i;
```

When expanding a node, $g(n)$ increments by 1 per step (unweighted social connections), and $h(n)$ is looked up from the precomputed BFS distance array:

```
1 double newG = gCost[curr.id] + 1;
2 double hVal = (hDist[next] == -1) ? 0 : hDist[next];
```

If `hDist[next] == -1`, the goal is unreachable from that node, so h defaults to 0 (safe fallback).

- **Path Reconstruction - *PathNode** buildPath** Once the goal is reached, the path is reconstructed by following `parent[]` pointers backwards from goal to start, inserting each node at the *head* of the linked list to produce the correct start-to-goal order:

```
1 newNode->next = head;
2 head = newNode;
3 if (curr == start) break;
4 curr = parent[curr];
```

```

1 //Initialize AStarNode
2 struct AStarNode1 {
3     int id;
4     double f, g, h;
5     int parent;
6 };
7
8 PathNode* buildPath(const vector<int> &parent, const vector<
    double> &gCost, const vector<double> &hCost, int start, int
    goal){
9     PathNode *head = nullptr;
10    int curr = goal;
11    while (curr != -1){
12        PathNode *newNode = new PathNode();
13        newNode->name = to_string(curr);
14        newNode->g = gCost[curr];
15        newNode->h = hCost[curr];
16        newNode->f = gCost[curr] + hCost[curr];
17        newNode->next = head;
18        head = newNode;
19        if (curr == start) break;
20        curr = parent[curr];
21    }
22    return head;
23 }
24 vector<int> calcHeuristic(double adjMatrix[100][100], int goal,
    int n) {
25     vector<int> dist(n, -1);
26     vector<int> bfsQueue;
27
28     dist[goal] = 0;
29     bfsQueue.push_back(goal);
30
31     int idx = 0;
32     while (idx < (int)bfsQueue.size()) {
33         int curr = bfsQueue[idx++];
34         for (int next = 0; next < n; next++) {
35
36             if (adjMatrix[next][curr] != 0 && dist[next] == -1)
37             {
38                 dist[next] = dist[curr] + 1;
39                 bfsQueue.push_back(next);
40             }
41         }
42     }
43     return dist;
44 }
45 //Task 1
46 PathNode* findSocialPath(double adjMatrix[100][100], int
    startPerson, int goalPerson){
47     int n = 100;

```

```

47     vector<double> gCost(n,1e9);
48     vector<double> hCost(n,0);
49     vector<bool> closed(n, false);
50     vector<int> parent(n, -1);
51     vector<AStarNode1> openList;
52
53     vector<int> hDist = calcHeuristic(adjMatrix, goalPerson, n);
54
55     double hStart = (hDist[startPerson] == -1) ? 0 : hDist[
        startPerson];
56     gCost[startPerson] = 0;
57     hCost[startPerson] = hStart;
58     openList.push_back({startPerson, hStart, 0, hStart, -1});
59     while (!openList.empty()){
60         int bestIdx = 0;
61         for (int i = 1; i < openList.size(); ++i){
62             if (openList[i].f < openList[bestIdx].f){
63                 bestIdx = i;
64             }
65         }
66         AStarNode1 curr = openList[bestIdx];
67         openList.erase(openList.begin() + bestIdx);
68
69         if (curr.id == goalPerson) return buildPath(parent,
            gCost, hCost, startPerson, goalPerson);
70         closed[curr.id] = true;
71         for (int next = 0; next < n; ++next){
72             if (adjMatrix[curr.id][next] == 0 || closed[next])
73                 continue;
74             double newG = gCost[curr.id] + 1;
75             if (newG < gCost[next]){
76                 gCost[next] = newG;
77                 parent[next] = curr.id;
78                 double hVal = (hDist[next] == -1) ? 0 : hDist[
                    next];
79                 hCost[next] = hVal;
80                 openList.push_back({next, newG+hVal, newG, hVal,
                    curr.id});
81             }
82         }
83     }
84     return nullptr;

```

Listing 4: Heuristic Function

Following to the above example: we have result for the case *Alice* -> *David*

```

1 double adj[100][100] = {};
2
3 // Social Network - 5 people:
4 // 0-Alice, 1-Bob, 2-Charlie, 3-David, 4-Eve
5 //
6 // Alice -- Bob -- Charlie -- David
7 //                      |
8 //                      Eve
9
10 adj[0][1] = adj[1][0] = 1; // Alice - Bob
11 adj[1][2] = adj[2][1] = 1; // Bob - Charlie
12 adj[2][3] = adj[3][2] = 1; // Charlie - David
13 adj[2][4] = adj[4][2] = 1; // Charlie - Eve
14
15 cout << "=== Alice -> David ===\n";
16 PathNode* r1 = findSocialPath(adj, 0, 3);
17 printPath(r1);

```

Output

```

1 =====
2           TASK 1 TEST
3 =====
4 === Alice -> David ===
5 Node: 0 | f: 3 | g: 0 | h: 3
6 Node: 1 | f: 3 | g: 1 | h: 2
7 Node: 2 | f: 3 | g: 2 | h: 1
8 Node: 3 | f: 3 | g: 3 | h: 0
9
10 === Alice -> Eve ===
11 Node: 0 | f: 3 | g: 0 | h: 3
12 Node: 1 | f: 3 | g: 1 | h: 2
13 Node: 2 | f: 3 | g: 2 | h: 1
14 Node: 4 | f: 3 | g: 3 | h: 0

```

Listing 5: Actual program output

2.2 Task 2 — Drone Delivery in 2D Space with Three Heuristic

2.2.1 Problem Overview

To evaluate the mathematical behavior and demonstrate the specific advantages of each heuristic mode, we perform a manual trace using a 4-node delivery network. The spatial coordinates $[X, Y]$ of the nodes are defined as follows:

- Node 0: (0, 0) (Start Point)
- Node 1: (2, 0)
- Node 2: (2, 3)
- Node 3: (5, 3) (Goal Point)

The non-zero edge weights in the network adjacency matrix (`adjMatrix`) are:

- $W(0 \rightarrow 1) = 3$
- $W(0 \rightarrow 2) = 8$
- $W(1 \rightarrow 2) = 4$
- $W(1 \rightarrow 3) = 7$
- $W(2 \rightarrow 3) = 2$

-
- Value Calculations ($h(n)$ to Goal Node 3)

Before executing the search loops, the distance from each node n to the Goal Node 3 (5, 3) is calculated for all three modes using their respective geometric formulas:

Table 1: Pre-computed Heuristic Values for Mode 1 (Manhattan)

Node	Coordinates	Mode 1: $h(n) = x_n - x_{goal} + y_n - y_{goal} $
0	(0, 0)	$ 0 - 5 + 0 - 3 = \mathbf{8}$
1	(2, 0)	$ 2 - 5 + 0 - 3 = \mathbf{6}$
2	(2, 3)	$ 2 - 5 + 3 - 3 = \mathbf{3}$
3	(5, 3) (Goal)	$ 5 - 5 + 3 - 3 = \mathbf{0}$

Table 2: Pre-computed Heuristic Values for Mode 2 (Euclidean)

Node	Coordinates	Mode 2: $h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$
0	(0, 0)	$\sqrt{(0 - 5)^2 + (0 - 3)^2} = \sqrt{34} \approx \mathbf{5.83095}$
1	(2, 0)	$\sqrt{(2 - 5)^2 + (0 - 3)^2} = \sqrt{18} \approx \mathbf{4.24264}$
2	(2, 3)	$\sqrt{(2 - 5)^2 + (3 - 3)^2} = \sqrt{9} = \mathbf{3.00000}$
3	(5, 3) (Goal)	$\sqrt{(5 - 5)^2 + (3 - 3)^2} = \mathbf{0.00000}$

Table 3: Pre-computed Heuristic Values for Mode 3 (Chebyshev)

Node	Coordinates	Mode 3: $h(n) = \max(x_n - x_{goal} , y_n - y_{goal})$
0	(0, 0)	$\max(0 - 5 , 0 - 3) = 5$
1	(2, 0)	$\max(2 - 5 , 0 - 3) = 3$
2	(2, 3)	$\max(2 - 5 , 3 - 3) = 3$
3	(5, 3) (Goal)	$\max(5 - 5 , 3 - 3) = 0$

2.2.2 Source Code

In this task, the node *AStarNode1* is used and not declared again.

```

1 //The function getHeuristic is declared above
2 PathNode* buildPath2(const vector<int> &parents, const vector<
  double> &gCost, const vector<double> &hCost, int start, int
  goal, int coords[100][2]){
3     PathNode *head = nullptr;
4     int curr = goal;
5     while (curr != -1){
6         PathNode *newNode = new PathNode();
7         newNode->name = "(" + to_string(coords[curr][0]) + "," +
          to_string(coords[curr][1]) + ")";
8         newNode->g = gCost[curr];
9         newNode->h = hCost[curr];
10        newNode->f = gCost[curr] + hCost[curr];
11        newNode->next = nullptr;
12
13        newNode->next = head;
14        head = newNode;
15        if (curr == start) break;
16        curr = parents[curr];
17
18    }
19    return head;
20 }
21 PathNode* findDronePath(double adjMatrix[100][100], int coords
  [100][2], int startPoint, int goalPoint, int mode){
22     int n = 100;
23     vector<int> parent(n, -1);
24     vector<double> gCost(n, 1e9);
25     vector<double> hCost(n, 0);
26     vector<AStarNode1> openList;
27     vector<bool> closed(n, false);
28     double hStart = getHeuristic(mode, coords[startPoint][0],
        coords[goalPoint][0], coords[startPoint][1], coords[
        goalPoint][1]);
29     hCost[startPoint] = hStart;
30     gCost[startPoint] = 0;
31     openList.push_back({startPoint, hStart, 0, hStart, -1});
32     while (!openList.empty()){
33         int bestIdx = 0;

```

```

34 for (int i = 1; i < openList.size(); ++i) {
35     if (openList[i].f < openList[bestIdx].f) {
36         bestIdx = i;
37     }
38
39     else if (openList[i].f == openList[bestIdx].f &&
40             openList[i].g > openList[bestIdx].g) {
41         bestIdx = i;
42     }
43 }
44
45 AStarNode1 curr = openList[bestIdx];
46 openList.erase(openList.begin() + bestIdx);
47 if (closed[curr.id]) continue;
48 if (curr.id == goalPoint) return buildPath2(parent,
49     gCost, hCost, startPoint, goalPoint, coords);
50 closed[curr.id] = true;
51 for (int next = 0; next < n; ++next){
52     if (adjMatrix[curr.id][next] == 0 || closed[next])
53         continue;
54     double newG = adjMatrix[curr.id][next] + gCost[curr.
55         id];
56     if (newG < gCost[next]){
57         gCost[next] = newG;
58         parent[next] = curr.id;
59         double hVal = getHeuristic(mode, coords[next
60             ][0], coords[goalPoint][0], coords[next][1],
61             coords[goalPoint][1]);
62         hCost[next] = hVal;
63         openList.push_back({next, newG+hVal, newG, hVal,
64             curr.id});
65     }
66 }
67
68 return nullptr;
69 }

```

2.2.3 Step-by-Step Manual Trace

Since the edge weights are identical across all scenarios, the expansion order for this specific graph remains uniform, but the evaluation cost metrics ($f = g + h$) differ.

- Execution Loop Trace
- **Initialization:** Populate $\text{openList} = \{\text{Node } 0\}$. Set $g(0) = 0$.
- **Iteration 1:**
 - Pop the best node: **Node 0**.
 - Expand Node 0's neighbors: Node 1 and Node 2.

- **Node 1:** $g(1) = g(0) + W(0 \rightarrow 1) = 0 + 3 = 3$. $\text{Parent}(1) = 0$.
 - **Node 2:** $g(2) = g(0) + W(0 \rightarrow 2) = 0 + 8 = 8$. $\text{Parent}(2) = 0$.
 - `openList` updates to $\{\text{Node 1}, \text{Node 2}\}$.
 - **Iteration 2:** Evaluate current f -costs in the `openList` to select the next step:
 - **Mode 1:** $f(1) = 3 + 6 = 9$ vs. $f(2) = 8 + 3 = 11 \rightarrow \text{Pop Node 1}$.
 - **Mode 2:** $f(1) = 3 + 4.24 = 7.24$ vs. $f(2) = 8 + 3.0 = 11.0 \rightarrow \text{Pop Node 1}$.
 - **Mode 3:** $f(1) = 3 + 3 = 6$ vs. $f(2) = 8 + 3 = 11 \rightarrow \text{Pop Node 1}$.
- Expand Node 1's neighbors: Node 2 and Node 3 (Goal).
- **Node 2:** New $g(2) = g(1) + W(1 \rightarrow 2) = 3 + 4 = 7$. Since $7 < 8$ (the initial cost via Node 0), update $g(2) = 7$ and change its link state $\text{Parent}(2) = 1$.
 - **Node 3:** $g(3) = g(1) + W(1 \rightarrow 3) = 3 + 7 = 10$. $\text{Parent}(3) = 1$.
 - `openList` updates to $\{\text{Node 2}, \text{Node 3}\}$.
 - **Iteration 3:** Evaluate current f -costs to choose the minimum:
 - **Mode 1:** $f(2) = 7 + 3 = 10$ vs. $f(3) = 10 + 0 = 10$. Under the code's tie-breaking policy, when f scores are equal, the node with the higher g -cost is preferred ($g(3) = 10 > g(2) = 7$). Thus, **Node 3** is popped. Goal reached!
 - **Mode 2:** $f(2) = 7 + 3.0 = 10.0$ vs. $f(3) = 10 + 0 = 10.0$. Due to the higher g -cost rule ($g(3) > g(2)$), **Node 3** is popped. Goal reached!
 - **Mode 3:** $f(2) = 7 + 3 = 10$ vs. $f(3) = 10 + 0 = 10$. Popping **Node 3** based on the higher g -cost tie-breaking metric. Goal reached!

2.2.4 Final Reconstructed Output Path

The execution results obtained from the actual compiled C++ program match the manual calculations perfectly:

```

1  === Task 2: Mode 1 (Manhattan) ===
2  Node: (0,0) | f: 8 | g: 0 | h: 8
3  Node: (2,0) | f: 9 | g: 3 | h: 6
4  Node: (5,3) | f: 10 | g: 10 | h: 0
5
6  === Task 2: Mode 2 (Euclidean) ===
7  Node: (0,0) | f: 5.83095 | g: 0 | h: 5.83095
8  Node: (2,0) | f: 7.24264 | g: 3 | h: 4.24264
9  Node: (5,3) | f: 10 | g: 10 | h: 0
10
11 === Task 2: Mode 3 (Chebyshev) ===
12 Node: (0,0) | f: 5 | g: 0 | h: 5
13 Node: (2,0) | f: 6 | g: 3 | h: 3
14 Node: (5,3) | f: 10 | g: 10 | h: 0

```

Listing 6: Actual program output

2.2.5 Analytical Comparison: Structural Advantages of Each Heuristic Mode

- **Advantage of Mode 1 (Manhattan Distance)**

- **Mathematical Property:** $h(n) = |x_1 - x_2| + |y_1 - y_2|$. It assumes movement is restricted exclusively to orthogonal grid lines (4-directional grid).
- **Drone Operational Advantage:**
 - * **Core Advantage:** Perfectly models **obstacle-heavy, grid-like environments** (e.g., dense urban areas with tall skyscrapers).
 - * **Reason:** When drones are legally or physically restricted to fly only along orthogonal streets/corridors (4-directional movement), Manhattan acts as a perfectly informed, admissible estimator that accurately accounts for mandatory right-angle turns.

- **Advantage of Mode 2 (Euclidean Distance)**

- **Mathematical Property:** $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. It calculates the direct geometric straight line connecting two continuous vectors.
- **Drone Operational Advantage:**
 - * **Core Advantage:** Most accurate for **free-space, unconstrained flight** (e.g., rural areas, open airspaces, or high-altitude fields).
 - * **Reason:** Real aircraft naturally fly along direct geometric straight lines. The Euclidean metric perfectly mirrors this physical displacement, avoiding overestimations and minimizing unnecessary node evaluations to guarantee optimal paths.

- **Advantage of Mode 3 (Chebyshev Distance)**

- **Mathematical Property:** $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$. It models environments where diagonal steps cost exactly the same as horizontal or vertical steps (8-directional grid).
- **Drone Operational Advantage:**
- **Core Advantage:** Ideal for **omnidirectional drones with 8-directional mobility** where diagonal movement is highly efficient.
- **Reason:** It treats diagonal and orthogonal steps equally. As seen in our trace, both Node 1 and Node 2 share an identical Chebyshev distance ($h = 3$) to Node 3, correctly recognizing that cutting diagonally satisfies multi-axis movement simultaneously without extra energy or time penalties.

2.3 Task 3 — Warehouse Robot Navigation with Obstacles

2.3.1 Problem Overview

Given a warehouse as an $m \times n$ grid where each cell is either **passable** (0) or **blocked** (1), find the shortest path for a robot from a start coordinate to a goal coordinate using A*. The robot supports **8-directional movement**:

Direction	dx	dy	Cost
Up, Down, Left, Right	± 1 or 0	0 or ± 1	1.0
Up-Left, Up-Right, Down-Left, Down-Right	± 1	± 1	1.5

- $g(n)$: total movement cost from start to current cell
- $h(n)$: estimated cost to goal using the selected heuristic
- $f(n) = g(n) + h(n)$

Each node in the output path is named after the **direction taken** to reach that cell (e.g. “Up-Right”), along with f , g , h values.

2.3.2 Heuristic Modes

Let current cell = (x_1, y_1) , goal = (x_2, y_2) :

$$h_1(n) = |x_1 - x_2| + |y_1 - y_2| \quad (\text{Mode 1 — Manhattan})$$

$$h_2(n) = \max(|x_1 - x_2|, |y_1 - y_2|) \quad (\text{Mode 2 — Chebyshev})$$

Both are **admissible** — they never overestimate the true cost — so A* is guaranteed to return an optimal solution.

2.3.3 Manual Example Run

Warehouse Grid (5×5)

0	0	0	0	0
0	W	W	0	0
0	0	0	W	0
0	W	0	0	0
S	0	0	0	G

(W = wall, S = start (4,0), G = goal (0,4))

```

1  int maze[100][100] = {
2      {0,0,0,0,0},
3      {0,1,1,0,0},
4      {0,0,0,1,0},
5      {0,1,0,0,0},
6      {0,0,0,0,0}
7  };
8  // Mode 1
9  PathNode* r1 = findPathInMaze(maze, 5, 5, 4, 0, 0, 4, 1);
10 printPath(r1);
11 // Mode 2

```

```

12 PathNode* r2 = findPathInMaze(maze, 5, 5, 4, 0, 0, 4, 2);
13 printPath(r2);

```

Listing 7: Maze definition and test in main.cpp

Heuristic Values at Start (4,0), Goal (0,4)

Heuristic	Formula	Value at start
Mode 1 (Manhattan)	$ 4 - 0 + 0 - 4 $	$4 + 4 = 8$
Mode 2 (Chebyshev)	$\max(4 - 0 , 0 - 4)$	$\max(4, 4) = 4$

Mode 1 — Manhattan: Step-by-Step

$$h(x, y) = |x - 0| + |y - 4|$$

Step	Direction	Cell	g	h	f
1	Up	(3, 0)	1.0	7	8.0
2	Up-Right	(2, 1)	2.5	5	7.5
3	Right	(2, 2)	3.5	4	7.5
4	Up-Right	(1, 3)	5.0	2	7.0
5	Up-Right	(0, 4)	6.5	0	6.5

Total cost: 6.5 Path: $(4, 0) \rightarrow (3, 0) \rightarrow (2, 1) \rightarrow (2, 2) \rightarrow (1, 3) \rightarrow (0, 4)$

Why not go diagonal immediately? From (4, 0), going Up-Right would reach (3, 1) — but that path is eventually blocked by the wall at (3, 1), so A* navigates Up first, then diagonally around the obstacles.

Mode 2 — Chebyshev: Step-by-Step

$$h(x, y) = \max(|x - 0|, |y - 4|)$$

Step	Direction	Cell	g	h	f
1	Up	(3, 0)	1.0	4	5.0
2	Up-Right	(2, 1)	2.5	3	5.5
3	Right	(2, 2)	3.5	2	5.5
4	Up-Right	(1, 3)	5.0	1	6.0
5	Up-Right	(0, 4)	6.5	0	6.5

Total cost: 6.5 Path: $(4, 0) \rightarrow (3, 0) \rightarrow (2, 1) \rightarrow (2, 2) \rightarrow (1, 3) \rightarrow (0, 4)$

Program Output

```

1 === Mode 1 (Manhattan) ===
2 Solution Path:
3 Node: Up          | f: 8.0 | g: 1.0 | h: 7
4 Node: Up-Right    | f: 7.5 | g: 2.5 | h: 5
5 Node: Right       | f: 7.5 | g: 3.5 | h: 4
6 Node: Up-Right    | f: 7.0 | g: 5.0 | h: 2
7 Node: Up-Right    | f: 6.5 | g: 6.5 | h: 0

```

```

8
9 === Mode 2 (Chebyshev) ===
10 Solution Path:
11 Node: Up      | f: 5.0 | g: 1.0 | h: 4
12 Node: Up-Right | f: 5.5 | g: 2.5 | h: 3
13 Node: Right   | f: 5.5 | g: 3.5 | h: 2
14 Node: Up-Right | f: 6.0 | g: 5.0 | h: 1
15 Node: Up-Right | f: 6.5 | g: 6.5 | h: 0

```

Listing 8: Actual program output

Comparison of Both Heuristic Modes

	Mode 1 (Manhattan)	Mode 2 (Chebyshev)
Initial h at start	8	4
Total path cost	6.5	6.5
Number of steps	5	5
Path found	identical	identical

- Both modes find the same optimal path with total cost 6.5, demonstrating that both heuristics are admissible and correct in this scenario.
- **Mode 1 (Manhattan)** starts with $h = 8$, a higher estimate that causes A* to be more conservative. The f value decreases steadily as the robot approaches the goal, reflecting the diminishing Manhattan distance.
- **Mode 2 (Chebyshev)** starts with $h = 4$, a tighter lower bound that better reflects the diagonal movement capability of the robot. This leads A* to open fewer unnecessary nodes during the search, as the estimate more closely matches the actual remaining cost through diagonal moves.
- The **obstacle layout** forces both modes through the same corridor: Up to (3, 0), diagonal to (2, 1), Right to (2, 2) (bypassing the wall at (2, 3)), then diagonal to the goal. This demonstrates that the A* algorithm correctly handles obstacles regardless of which heuristic mode is selected.

2.3.4 Source Code

- Key Design Decisions
 - A **2D array** is used for `gCost`, `hCost`, `closed`, `parentX`, `parentY`, and `dirMap` — one entry per grid cell — instead of a 1D vector indexed by a flattened ID, to keep coordinate-based access intuitive.
 - The **closed check** is performed both when a neighbor is discovered and when a node is popped from the open list, to handle duplicate entries that arise when a shorter path to a node is found after it has already been enqueued.
 - **Path reconstruction** inserts each node at the *head* of the linked list while tracing `parent` pointers backwards from goal to start, producing the correct start-to-goal order without an explicit reverse step.

```

1 double getHeuristicMaze(int mode, int x1, int x2, int y1, int y2
2 ) {
3     if (mode == 1) return abs(x1-x2) + abs(y1-y2);
4     if (mode == 2) return max(abs(x1-x2), abs(y1-y2));
5     return 0;
6 }
7
8 PathNode *buildPath3(int pX[100][100], int pY[100][100], string
9 dirName[100][100], double gCost[100][100], double hCost
10 [100][100], int startX, int startY, int goalX, int goalY) {
11     PathNode* head = nullptr;
12     int gx = goalX, gy = goalY;
13
14     while (true) {
15
16         if (gx == startX && gy == startY) break;
17
18         PathNode* newNode = new PathNode();
19         newNode->name      = dirName[gx][gy];
20         newNode->g          = gCost[gx][gy];
21         newNode->h          = hCost[gx][gy];
22         newNode->f          = gCost[gx][gy] + hCost[gx][gy];
23         newNode->next       = head;
24         head               = newNode;
25
26         int px = pX[gx][gy], py = pY[gx][gy];
27         gx = px;
28         gy = py;
29     }
30     return head;
31 }
32
33 PathNode* findWarehousePath(int warehouse[100][100], int m, int
34 n, int startX, int startY, int goalX, int goalY, int mode) {
35     int pX[100][100], pY[100][100];
36     bool closed[100][100];
37     double gCost[100][100];
38     double hCost[100][100];
39     string dirName[100][100];
40     for (int i = 0; i < m; ++i) {
41         for (int j = 0; j < n; ++j) {
42             pX[i][j] = -1;
43             pY[i][j] = -1;
44             closed[i][j] = false;
45             gCost[i][j] = 1e9;
46             hCost[i][j] = 0;
47             dirName[i][j] = "";
48         }
49     }
50     vector<AStarNode2> openList;
51     double hStart = getHeuristicMaze(mode, startX, goalX, startY, goalY);

```

```

    , goalY);
48  hCost[startX][startY] = hStart;
49  gCost[startX][startY] = 0;
50  openList.push_back({startX, startY, 0, hStart, hStart,
    -1,-1,"Start"});
51  while (!openList.empty()){
52      int bestIdx = 0;
53      for (int i = 1; i < openList.size(); ++i){
54          if (openList[i].f < openList[bestIdx].f){
55              bestIdx = i;
56          }
57
58      }
59      AStarNode2 curr = openList[bestIdx];
60      openList.erase(openList.begin() + bestIdx);
61      if (closed[curr.x][curr.y]) continue;
62      if (curr.x == goalX && curr.y == goalY) return
        buildPath3(pX, pY, dirName, gCost, hCost, startX,
        startY, goalX, goalY);
63      closed[curr.x][curr.y] = true;
64      for (auto &dir : dirs){
65          int nX = curr.x + dir.dx;
66          int nY = curr.y + dir.dy;
67          if (nX < 0 || nX >= m || nY < 0 || nY >= n ||
            warehouse[nX][nY] == 1 || closed[nX][nY]) continue
            ;
68          double newG = gCost[curr.x][curr.y] + dir.cost;
69          if (newG < gCost[nX][nY]){
70              gCost[nX][nY] = newG;
71              pX[nX][nY] = curr.x;
72              pY[nX][nY] = curr.y;
73              dirName[nX][nY] = dir.name;
74
75              double hVal = getHeuristicMaze(mode, nX, goalX,
                nY, goalY);
76              hCost[nX][nY] = hVal;
77              openList.push_back({nX, nY, newG, hVal, hVal +
                newG, curr.x, curr.y, dir.name});
78          }
79
80      }
81  }
82  }
83  return nullptr;
84 }

```

Listing 9: findPathInMaze — Task 3 implementation

2.4 Task 4 — Evacuation Route Planning

2.4.1 Problem Overview

Given a building floor plan as an $m \times n$ grid, the task requires two steps:

1. **Convert** the grid into a weighted adjacency matrix where each passable cell is a vertex and each valid move is a directed edge with cost 1.0 (cardinal) or 1.5 (diagonal).
2. **Run A*** on the weight matrix to find the shortest evacuation route from start to exit.
 - $g(n)$: total movement cost from start to current vertex
 - $h(n)$: estimated cost to exit (Mode 1: Manhattan, Mode 2: Chebyshev)
 - $f(n) = g(n) + h(n)$
 - Output: linked list path **and** the filled weight matrix

2.4.2 Grid to Weight Matrix Conversion

Each cell (x, y) in an $m \times n$ grid is assigned a unique index:

$$\text{index} = x \times n + y$$

For every passable cell, all 8 neighbors are examined. If a neighbor is in-bounds and passable:

$$W[\text{idx}(x, y)][\text{idx}(x+dx, y+dy)] = \begin{cases} 1.0 & \text{cardinal move (Up/Down/Left/Right)} \\ 1.5 & \text{diagonal move} \end{cases}$$

Wall cells are completely skipped — no edges to or from them.

2.4.3 Manual Example Run

- Plan Grid (5×5)

0(0)	1(1)	2(2)	3(3)	4(4)
5(5)	W (6)	W (7)	8(8)	9(9)
10(10)	11(11)	12(12)	W (13)	14(14)
15(15)	W (16)	17(17)	18(18)	19(19)
S (20)	21(21)	22(22)	23(23)	G (24)

(**W** = wall, **S** = start $(4, 0) = \text{idx } 20$, **G** = exit $(0, 4) = \text{idx } 4$, $n = 5$ columns)

```

1 int floorPlan[100][100] = {
2     {0,0,0,0,0},
3     {0,1,1,0,0},
4     {0,0,0,1,0},
5     {0,1,0,0,0},

```



```

6      {0,0,0,0,0}
7  };
8  double W[100][100] = {};
9
10 // Mode 1
11 PathNode* r1 = findEvacuationPath(floorPlan,5,5,4,0,0,4,W,1);
12 printPath(r1);
13
14 // Mode 2
15 PathNode* r2 = findEvacuationPath(floorPlan,5,5,4,0,0,4,W,2);
16 printPath(r2);

```

Listing 10: Floor plan and test in main.cpp

Weight Matrix Construction (selected entries)

The full 25×25 weight matrix is built from the grid. Key entries along the evacuation path are:

From idx	To idx	From cell	To cell	Cost
20	15	(4, 0)	(3, 0)	1.0 (Up)
15	10	(3, 0)	(2, 0)	1.0 (Up)
10	11	(2, 0)	(2, 1)	1.0 (Right)
11	12	(2, 1)	(2, 2)	1.0 (Right)
12	8	(2, 2)	(1, 3)	1.5 (Up-Right)
8	4	(1, 3)	(0, 4)	1.5 (Up-Right)
(wall cells 6,7,13,16 have no edges)				

Heuristic Values at Start idx=20, (4, 0), Exit (0, 4)

Heuristic	Formula	Value
Mode 1 (Manhattan)	$ 4 - 0 + 0 - 4 $	8
Mode 2 (Chebyshev)	$\max(4 - 0 , 0 - 4)$	4

Mode 1 — Manhattan: Step-by-Step

$h(\text{idx}) = |x - 0| + |y - 4|$, traversing the weight matrix.

Step	Direction	idx	Cell	g	h	f
1	Up	15	(3, 0)	1.0	7	8.0
2	Up-Right	11	(2, 1)	2.5	5	7.5
3	Right	12	(2, 2)	3.5	4	7.5
4	Up-Right	8	(1, 3)	5.0	2	7.0
5	Up-Right	4	(0, 4)	6.5	0	6.5

Total cost: 6.5 **Path:** idx $20 \rightarrow 15 \rightarrow 11 \rightarrow 12 \rightarrow 8 \rightarrow 4$

Mode 2 — Chebyshev: Step-by-Step

$$h(\text{idx}) = \max(|x - 0|, |y - 4|)$$

Step	Direction	idx	Cell	g	h	f
1	Up	15	(3, 0)	1.0	4	5.0
2	Up-Right	11	(2, 1)	2.5	3	5.5
3	Right	12	(2, 2)	3.5	2	5.5
4	Up-Right	8	(1, 3)	5.0	1	6.0
5	Up-Right	4	(0, 4)	6.5	0	6.5

Total cost: 6.5 **Path:** idx 20 $\rightarrow$ 15 $\rightarrow$ 11 $\rightarrow$ 12 $\rightarrow$ 8 $\rightarrow$ 4

Program Output

```

1 === Mode 1 (Manhattan) ===
2 Solution Path:
3 Node: Up          | f: 8.0 | g: 1.0 | h: 7
4 Node: Up-Right    | f: 7.5 | g: 2.5 | h: 5
5 Node: Right       | f: 7.5 | g: 3.5 | h: 4
6 Node: Up-Right    | f: 7.0 | g: 5.0 | h: 2
7 Node: Up-Right    | f: 6.5 | g: 6.5 | h: 0
8
9 === Mode 2 (Chebyshev) ===
10 Solution Path:
11 Node: Up          | f: 5.0 | g: 1.0 | h: 4
12 Node: Up-Right    | f: 5.5 | g: 2.5 | h: 3
13 Node: Right       | f: 5.5 | g: 3.5 | h: 2
14 Node: Up-Right    | f: 6.0 | g: 5.0 | h: 1
15 Node: Up-Right    | f: 6.5 | g: 6.5 | h: 0

```

Listing 11: Actual program output

Comparison of Both Heuristic Modes

	Mode 1 (Manhattan)	Mode 2 (Chebyshev)
Initial h at start	8	4
Total path cost	6.5	6.5
Number of steps	5	5
Path (by index)	20 $\rightarrow$ 15 $\rightarrow$ 11 $\rightarrow$ 12 $\rightarrow$ 8 $\rightarrow$ 4	20 $\rightarrow$ 15 $\rightarrow$ 11 $\rightarrow$ 12 $\rightarrow$ 8 $\rightarrow$ 4

- Both modes find the **same optimal path** with total cost 6.5, confirming both heuristics are admissible and the weight matrix is correctly built.
- **Mode 1 (Manhattan)** starts with $h = 8$. The higher estimate causes A* to be more aggressive about reducing f early, but the f values decrease unevenly (8.0, 7.5, 7.5, 7.0, 6.5) because Manhattan does not perfectly model diagonal movement cost.
- **Mode 2 (Chebyshev)** starts with $h = 4$, a tighter lower bound for 8-directional movement. The f values increase monotonically (5.0, 5.5, 5.5, 6.0, 6.5), which is

characteristic of Chebyshev when the path blends cardinal and diagonal steps.

- The **key difference from Task 3** is the additional `buildWeightMatrix` step: the grid is first flattened into a 25×25 adjacency matrix indexed by $x \times n + y$, and A* then operates on this matrix rather than the grid directly. Wall cells (indices 6, 7, 13, 16) produce zero-weight rows and columns, effectively removing them from the search space.

2.4.4 Implementation

• Grid-to-Matrix Conversion

The conversion iterates over every passable cell and its 8 neighbors, writing edge weights into a flat $mn \times mn$ matrix:

```
1 int fromIdx = x * n + y;
2 int toIdx   = nx * n + ny;
3 weightMatrix[fromIdx][toIdx] = dir.cost;
```

Wall cells are detected by `floorPlan[nx][ny] == 1` and skipped entirely, leaving their matrix rows and columns as all-zero.

• A* on the Weight Matrix

Unlike Task 3 which uses a 2D `closed[x][y]` array, Task 4 uses a 1D `closed[id]` vector since nodes are identified by their flattened index. The neighbor search iterates over all indices and checks `weightMatrix[curr][next] != 0` to find valid moves:

```
1 for (int next = 0; next < total; next++) {
2     if (weightMatrix[curr.id][next] == 0 || closed[next])
3         continue;
4     double newG = gCost[curr.id] + weightMatrix[curr.id][next];
5     ...
6 }
```

• Source Code

```
1 PathNode* buildPath4(const vector<int> &parent, const vector
  <double> &gCost, const vector<double> &hCost, vector<
  string> &dir, int start, int goal){
2     PathNode *head = nullptr;
3     int curr = goal;
4     while (curr != -1){
5         PathNode *newNode = new PathNode();
6         newNode->name = (curr == start) ? "Start" : dir[curr
          ];
7         newNode->g = gCost[curr];
8         newNode->h = hCost[curr];
9         newNode->f = gCost[curr] + hCost[curr];
10
11
12         newNode->next = head;
```

```

13     head = newNode;
14     if (curr == start) break;
15     curr = parent[curr];
16
17 }
18 return head;
19 }
20 void buildWeightedMatrix(int floorPlan[100][100], int m, int
    n, double weightMatrix[100][100]){
21     for (int i = 0; i < m*n; ++i){
22         for (int j = 0; j < m * n; ++j){
23             weightMatrix[i][j] = 0;
24         }
25     }
26     for (int i = 0; i < m; ++i){
27         for (int j = 0; j < n; ++j){
28             if (floorPlan[i][j] == 1) continue;
29             int fromIdx = i * n + j;
30             for (auto &dir : dirs){
31                 int nx = i + dir.dx;
32                 int ny = j + dir.dy;
33                 if (nx < 0 || nx >= m) continue;
34                 if (ny < 0 || ny >= n) continue;
35                 if (floorPlan[nx][ny] == 1) continue;
36                 int toIdx = nx * n + ny;
37                 weightMatrix[fromIdx][toIdx] = dir.cost;
38             }
39         }
40     }
41
42 }
43 PathNode* findEvacuationPath(int floorPlan[100][100], int m,
    int n, int startX, int startY, int exitX, int exitY,
    double weightMatrix[100][100], int mode){
44     int start = startX * n + startY;
45     int goal = exitX * n + exitY;
46     int total = m * n;
47     buildWeightedMatrix(floorPlan, m, n, weightMatrix);
48     vector<int> parent(total, -1);
49     vector<double> gCost(total, 1e9);
50     vector<double> hCost(total, 0);
51     vector<bool> closed(total, false);
52     vector<string> dirMap(total, "");
53     vector<AStarNode3> openList;
54
55     double hVal = getHeuristic(mode, startX, exitX, startY,
        exitY);
56     gCost[start] = 0;
57     hCost[start] = hVal;
58
59     openList.push_back({start, 0, hVal, hVal, -1, "Start"});

```

```

60     while (!openList.empty()){
61         int bestIdx = 0;
62         for (int i = 1; i < openList.size(); ++i){
63             if (openList[i].f < openList[bestIdx].f){
64                 bestIdx = i;
65             }
66         }
67         AStarNode3 curr = openList[bestIdx];
68         openList.erase(openList.begin() + bestIdx);
69         if (closed[curr.id]) continue;
70         if (curr.id == goal) return buildPath4(parent, gCost,
71             hCost, dirMap, start, goal);
72         closed[curr.id] = true;
73         for (int next = 0; next < total; ++next){
74             if (weightMatrix[curr.id][next] == 0 || closed[next
75                 ]) continue;
76             double newG = weightMatrix[curr.id][next] + gCost[
77                 curr.id];
78             if (newG < gCost[next]){
79                 gCost[next] = newG;
80                 parent[next] = curr.id;
81
82                 int nx = next / n, ny = next % n;
83                 double hVal = getHeuristic(mode, nx, exitX, ny,
84                     exitY);
85                 hCost[next] = hVal;
86                 string dirN = "";
87                 int cx = curr.id/n, cy = curr.id%n;
88                 for (auto &dir : dirs){
89                     if (nx == cx + dir.dx && ny == cy + dir.dy){
90                         dirN = dir.name;
91                     }
92                 }
93                 dirMap[next] = dirN;
94                 openList.push_back({next, newG, hVal, newG +
95                     hVal, curr.id, dirN});
96             }
97         }
98     }
99     return nullptr;
100 }

```

Listing 12: findEvacuationPath — Task 4 implementation

2.4.5 The advantages of implementation in Task 4

- **Evacuation applicability:** Due to its flexibility, Task 4 is a powerful premise for the method which is designed for emergency escape route planning, where fast and reliable pathfinding is critical.
- **Edge-weighted graph modeling:** Converting the floor plan into a weighted adjacency matrix ($mn \times mn$) allows the A* algorithm to operate on a unified graph representation, separating the map data from the search logic.
- **Computational efficiency:** Using A* with admissible heuristics (Manhattan for Mode 1, Chebyshev for Mode 2) significantly reduces search time compared to naive algorithms, making real-time evacuation decisions feasible.
- **Optimality guarantee:** Both heuristic modes never overestimate the true movement cost, ensuring that the found path is always optimal under the given cost model (1.0 for cardinal moves, 1.5 for diagonal moves).

References

1. P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.
2. Wikipedia, “A* search algorithm,” https://en.wikipedia.org/wiki/A*_search_algorithm.
3. Geeksforgeeks, “A* Search Algorithm” <https://www.geeksforgeeks.org/dsa/a-search-algorithm/>.
4. Thanh Sach Le, *Introduction to Artificial Intelligence Course*
5. Marta Borowska-Stefańska, Michał Kowalski, Filip Turoboś, and Szymon Wiśniewski, “On determining the weight of edges in map-representing graphs-applications of heuristic methods in planning escape routes,” *Journal of Traffic and Transportation Engineering (English Edition)*, vol. 9, no. 6, pp. 1027–1043, 2022. DOI: [10.1016/j.jtte.2021.05.004](https://doi.org/10.1016/j.jtte.2021.05.004).